

Helm Paketmanagement

Agenda

1. Kubernetes

- [Architektur Kubernetes](#)
- [Aufbau von Browser zu Applikation - Schaubild](#)

2. Helm Einfuehrung

- [Was ist helm ?](#)
- [Was kann helm ?](#)
- [Was ist in helm ein chart?](#)
- [Warum Helm in Kubernetes verwenden ?](#)
- [Überblick über den Ablauf bei der Nutzung von helm \(Kommando: install\).](#)
- [Braucht helm das Programm kubectl ?](#)

3. Helm Installation und Konfiguration (inkl. kubectl)

- [Installation von kubectl unter Linux](#)
- [Konfiguration von kubectl mit namespaces](#)
- [Installation von helm unter Linux](#)
- [Installation bash completion](#)

4. Helm Grundlagen

- [TopLevel Objekte](#)

5. Helm - Spickzettel

- [Wichtig: Helm Spickzettel](#)

6. Arbeiten mit helm - charts

- [Installation, Upgrade, Uninstall helm-Chart exercise - simple \(mariadb-cloudpirates\).](#)
- [Nur fertiges manifest ausgeben ohne Installation](#)
- [Informationen aus nicht installierten Helm-Charts bekommen](#)
- [Chart runterladen und evtl. entpacken und bestimmte Version](#)
- [Aufräumen von CRD's nach dem Deinstallieren](#)

7. Helm Charts entwickeln

- [eigenes helm chart erstellen \(Gruppe\)](#)

8. Spezial: Umgang mit Einrückungen

- [Whitespaces meistern mit "-"](#)
- [Exercise Whitespaces](#)

9. Type - Conversions

- [Exercise toYaml](#)

10. Flow Control

- [if](#)
- [with](#)

- [range](#)
- 11. Named Templates
 - [named template](#)
 - [named template with dict](#)
- 12. Helm mit gitlab ci/cd
 - [Helm mit gitlab ci/cd ausrollen](#)
- 13. Metrics - Server
 - [Metrics - Server mit helm installieren und verwenden](#)
- 14. Helmfile
 - [Praxisbeispiel: openDesk mit Helmfile ausrollen](#)
- 15. helm - Dokumentation
 - [Helm Documentation](#)

Backlog

1. Grundlagen
 - [Feature / No-Features von Helm](#)
2. Tipps & Tricks
 - [kubernetes manifests mit privatem Repo](#)
 - [helm chart mit images auf privatem Repo](#)
3. Helm-Befehle und -Funktionen
 - [Repo einrichten](#)
 - [Suche in Repo und Artifacts Hub](#)
 - [Anzeigen von Informationen aus dem Chart von Online](#)
 - [Upgrade und auftretende Probleme](#)
4. Helm Repository
 - [Die wichtigsten Repo-Befehle](#)
5. Struktur von Helm - Charts
 - [Überblick](#)
6. Grundlagen Helm-Charts
 - [Testumgebung und Spaces \(2 Themen\)](#)
7. Erstellen von Helm-Charts
 - [Erstellen eines Guestbooks](#)
 - [Hooks für Guestbook erstellen](#)
 - [Dependencies/Abhängigkeiten herunterladen](#)
 - [Einfaches Testen](#)
 - [Input Validierung innerhalb von templates](#)
 - [Advanced Testing mit chart-testing](#)

- [Chart auf github veröffentlichen](#)

8. Sicherheit von helm-Chart

- [Grundlagen / Best Practices](#)
- [Security Encrypted Passwords in helm](#)

9. Testing in Helm-Charts

- [Testing in/von helm - charts](#)

10. Durchführung von Upgrades und Rollbacks von Anwendungen

11. Helm in Continuous Integration / Continuous Deployment (CI/CD) Pipelines

12. Tipps & Tricks

- [Set namespace in config of kubectl](#)
- [Create Ingress Redirect](#)
- [Helm Charts - Development - Best practices](#)

13. Integration mit anderen Tools

- [yamllint für Syntaxcheck von yaml - Dateien](#)

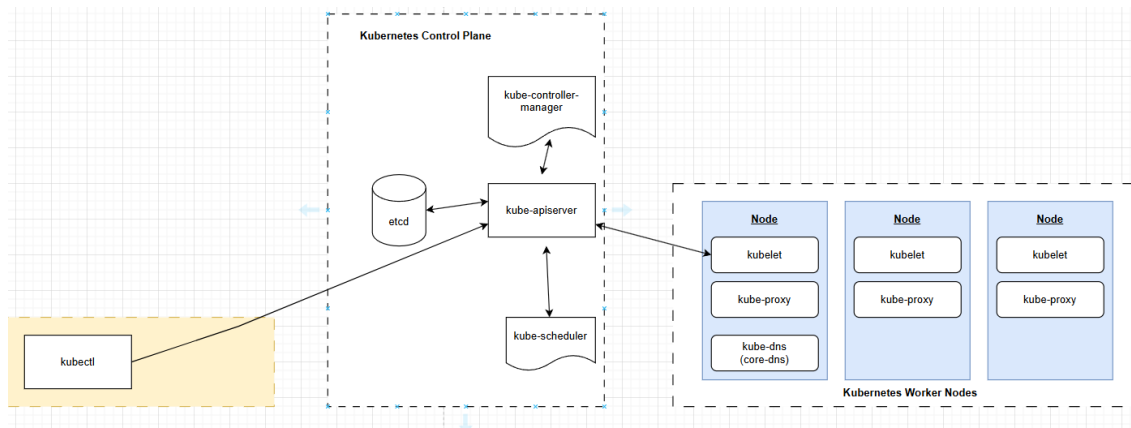
14. Troubleshooting und Debugging

- [helm template --validate - gegen api-server testen](#)

Kubernetes

Architektur Kubernetes

Schaubild



Komponenten / Grundbegriffe

Master (Control Plane)

Aufgaben

- Der Master koordiniert den Cluster
- Der Master koordiniert alle Aktivitäten in Ihrem Cluster
 - Planen von Anwendungen
 - Verwalten des gewünschten Status der Anwendungen
 - Skalieren von Anwendungen
 - Rollout neuer Updates.

Komponenten des Masters

ETCD

- Verwalten der Konfiguration des Clusters (key/value - pairs)

KUBE-CONTROLLER-MANAGER

- Zuständig für die Überwachung der Stati im Cluster mit Hilfe von endlos loops.
- kommuniziert mit dem Cluster über die kubernetes-api (bereitgestellt vom kube-api-server)

KUBE-API-SERVER

- provides api-frontend for administration (no gui)
- Exposes an HTTP API (users, parts of the cluster and external components communicate with it)
- REST API

KUBE-SCHEDULER

- assigns Pods to Nodes.
- scheduler determines which Nodes are valid placements for each Pod in the scheduling queue (according to constraints and available resources)
- The scheduler then ranks each valid Node and binds the Pod to a suitable Node.
- Reference implementation (other schedulers can be used)

Nodes

- Nodes (Knoten) sind die Arbeiter (Maschinen), die Anwendungen ausführen

- Ref: <https://kubernetes.io/de/docs/concepts/architecture/nodes/>

Pod/Pods

- Pods sind die kleinsten einsetzbaren Einheiten, die in Kubernetes erstellt und verwaltet werden können.
- Ein Pod (übersetzt Gruppe) ist eine Gruppe von einem oder mehreren Containern
 - gemeinsam genutzter Speicher- und Netzwerkressourcen
 - Befinden sich immer auf dem gleich virtuellen Server

Control Plane Node (former: master) - components

Node (Minion) - components

General

- On the nodes we will rollout the applications

kubelet

Node Agent that runs on every node (worker)

Er stellt sicher, dass Container in einem Pod ausgeführt werden.

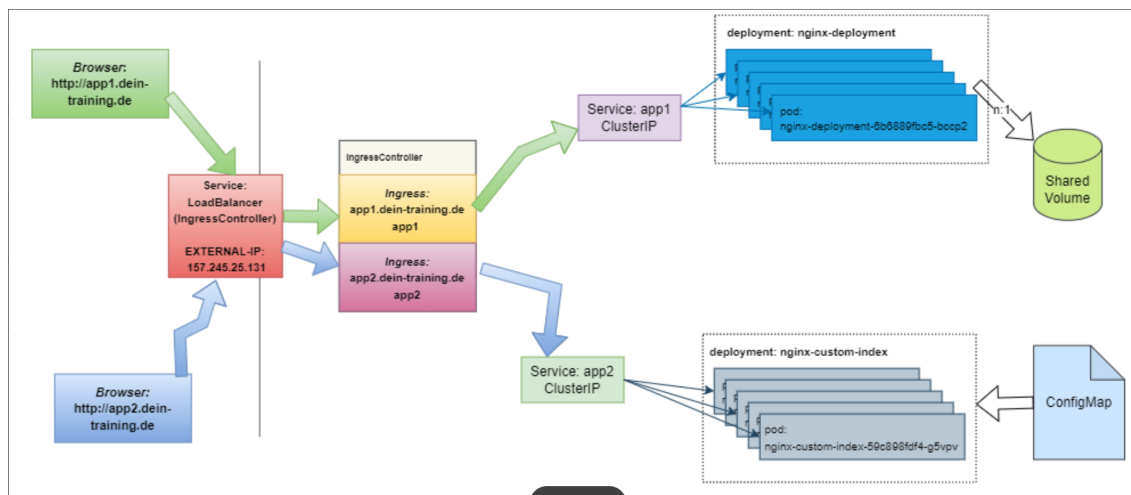
Kube-proxy

- Läuft auf jedem Node
- = Netzwerk-Proxy für die Kubernetes-Netzwerk-Services.
- Kube-proxy verwaltet die Netzwerkkommunikation innerhalb oder außerhalb Ihres Clusters.

Referenzen

- <https://www.redhat.com/de/topics/containers/kubernetes-architecture>

Aufbau von Browser zu Applikation - Schaubild



Helm Einfuehrung

Was ist helm ?

- Paketmanager für Kubernetes
- Ermöglicht Anwendungen in einem Kubernetes-Cluster zu definieren, zu installieren und zu verwalten

- ähnlich wie `apt` bei Debian oder `yum` bei CentOS, aber speziell für Kubernetes.

Was kann helm ?

- **Installieren** und **Deinstallieren** von Anwendungen in Kubernetes (`helm install` / `helm uninstall`)
- **Upgraden** von bestehenden Installationen (`helm upgrade`)
- **Rollbacks** durchführen, falls etwas schief läuft (`helm rollback`)
- **Anpassen** von Anwendungen durch Konfigurationswerte (`values.yaml`)
- **Veröffentlichen** eigener Charts (z. B. in einem Helm-Repository)

Was ist in helm ein chart?

Definition

- Ein **Helm Chart** ist ein Paket, das alle nötigen Kubernetes-Ressourcen beschreibt, um eine Anwendung oder einen Dienst bereitzustellen.

Es enthält:

- **Templates:** Vorlagen in YAML-Format, die dynamisch Werte einsetzen
- **values.yaml:** Eine Datei mit Konfigurationswerten
- **Chart.yaml:** Metainformationen zum Chart (Name, Version, etc.)
- **Abhängigkeiten:** Optional können andere Charts mit eingebunden werden

Formate / Ort

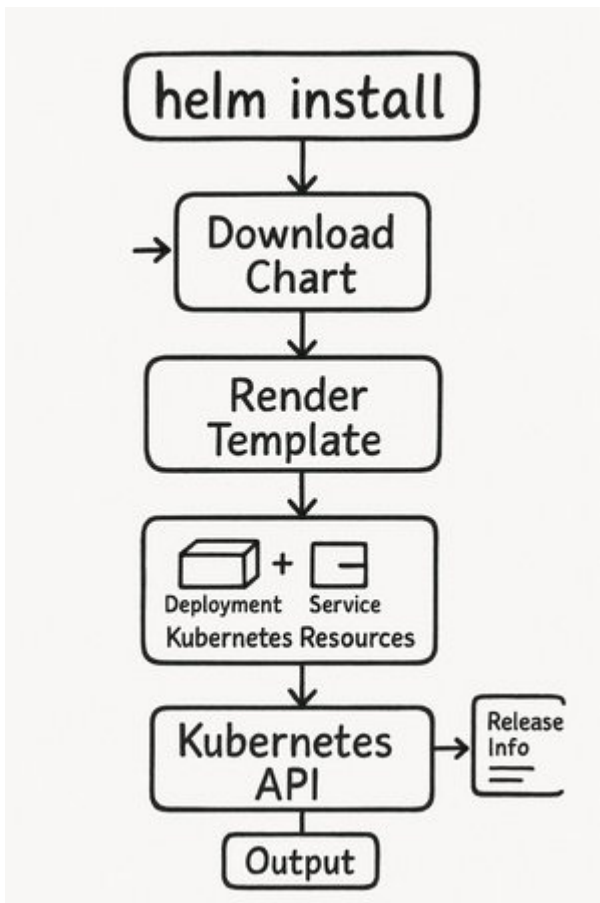
- Verzeichnis z.B. meine-app (und in dem Verzeichnis die bekannte Struktur von oben)
- tgz (Tape-Archive mit gzip komprimiert)
- URL

Warum Helm in Kubernetes verwenden ?

- **Wiederverwendbarkeit:** Ein Chart kann mehrfach und in unterschiedlichen Umgebungen genutzt werden.
- **Konfigurierbarkeit:** Anpassung an verschiedene Umgebungen wie Entwicklung, Test, Produktion.
- **Automatisierbarkeit:** Ideal für den Einsatz in CI/CD-Pipelines.
- **Große Community:** Viele fertige Charts für beliebte Software wie Prometheus, Grafana, nginx, etc.

Überblick über den Ablauf bei der Nutzung von helm (Kommando: install)

Grafik



Der Weg

Wenn der Befehl `helm install` ausgeführt wird, passiert intern Folgendes:

1. Chart-Abfrage:

- Helm sucht Chart lokal oder im Repos und lädt es herunter.

2. Chart-Templating:

- Helm rendert die Templates im Chart.
- Variablen werden (wie in der `values.yaml` definiert) in die Templates eingefügt.
- Dadurch werden manifeste für Kubernetes-Ressourcen (z. B. Deployments, Services) erstellt.

3. Kubernetes API:

- Das gerenderte Kubernetes Manifest wird an den Kubernetes-API geschickt.

4. Release-Verwaltung:

- Helm speichert die Chart- und Versionsinformationen in der Helm-Release-Datenbank (in Kubernetes als Secret)
- Dies ermöglicht eine spätere Verwaltung und Aktualisierung des Releases.

5. Ausgabe (templates/NOTES.txt):

- Helm gibt den Status des Installationsprozesses aus, einschließlich der erstellten Ressourcen und etwaiger Fehler.

Long story short

- Helm rendert Kubernetes-Ressourcen aus einem Chart und kommuniziert mit der Kubernetes-API, um diese Ressourcen zu erstellen und ein Release zu verwalten.

Braucht helm das Programm kubectl ?

- helm braucht zwar kubectl nicht, es verwendet aber auch die .kube/config - Datei per Default

Helm Installation und Konfiguration (inkl. kubectl)

Installation von kubectl unter Linux

Walkthrough (Start with unprivileged user like training or kurs)

```
sudo su -
```

```
## Get current version
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
## install the kubectl to the right directory
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Konfiguration von kubectl mit namespaces

config einrichten

```
cd
mkdir -p .kube
cd .kube
cp -a /tmp/config config
ls -la
## Alternative: nano config befüllen
## das bekommt ihr aus Eurem Cluster Management Tool
```

```
kubectl cluster-info
```

Arbeitsbereich konfigurieren

```
kubectl create ns jochen
kubectl get ns
kubectl config set-context --current --namespace jochen
kubectl get pods
```

Installation von helm unter Linux

Walkthrough (Start as unprivileged user, e.g. training or kurs)

```
sudo su -
```

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
```



```
chmod 700 get_helm.sh
./get_helm.sh
```

```
exit
```

Reference:

- <https://helm.sh/docs/intro/install/>

Installation bash completion

```
sudo su -
helm completion bash > /etc/bash_completion.d/helm
exit
## z.B.
su - tln11
```

Helm Grundlagen

TopLevel Objekte

.Chart

- Zieht alle Informationen aus der Chart.yaml
- Alle Eigenschaften fangen mit einem grossen Buchstaben, statt klein wie im Chart, z.B. .Chart.Name

.Values

- Ansprechen der Values bzw. Default Values

.Release

- Ansprechen aller Eigenschaften aus der Release z.B. Release.Name

Helm - Spickzettel

Wichtig: Helm Spickzettel

Alle helm-releases anzeigen

```
## im eigenen Namespace
helm list
## in allen Namespaces
helm list -A
## für einen speziellen
helm -n kube-system list
```

Helm - Chart installieren

```
## Empfehlung mit namespace
## Repo hinzufügen für Client
helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
helm install my-nginx bitnami/nginx --version 19.0.1 --create-namespace --  
namespace=app-<namenskuerzel>
```

Helm - Suche

```
## welche Repos sind konfiguriert  
helm repo list  
helm search repo bitnami  
helm search hub
```

Helm - template

```
## Rendern des Templates  
helm repo add bitnami https://charts.bitnami.com/bitnami  
helm template my-nginx bitnami/nginx  
helm template bitnami/nginx
```

Arbeiten mit helm - charts

Installation, Upgrade, Uninstall helm-Chart exercise - simple (mariadb-cloudpirates)

Schritt 1: install mariadb von cloudpirates

```
## Mini-Step 1: Testen  
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --  
reset-values --version 0.5.1 --dry-run=server
```

```
## Mini-Step 2: Installieren  
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --  
reset-values --version 0.5.1
```

```
## Geht das denn auch ?  
kubectl get pods
```

Schritt 2: Exercise: Upgrade to new version

Schritt 2.1 Default values (auf terminal) ausfindig machen

```
## Recherchiere wie die Werte gesetzt werden (artifactory.io) oder verwende die  
folgenden Befehle:  
helm show values oci://registry-1.docker.io/cloudpirates/mariadb  
helm show values oci://registry-1.docker.io/cloudpirates/mariadb | less
```

Schritt 2.2 Upgrade und resources ändern

```
cd  
mkdir -p mariadb-values  
cd mariadb-values
```

```
mkdir prod
cd prod
```

```
nano values.yaml
```

```
resources:
  limits:
    memory: 300Mi
  requests:
    memory: 300Mi
    cpu: 100m
```

```
cd ..
```

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.3 --dry-run -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.3 -f prod/values.yaml
```

```
kubectl get pods
```

Umschauen

```
kubectl get pods
## Ab Version 4 (helm) sinnvoll
helm status my-mariadb
helm list
## alle helm charts anzeigen, die im gesamten Cluster installiert wurden
helm list -A
helm history my-mariadb
```

Umschauen get

```
## Wo speichert er Information, die er später mit helm get abrufen
kubectl get secrets
```

```
helm get values my-mariadb
helm get manifest my-mariadb
## Zeile ausgeben und 4 Zeilen danach und 4 Zeilen davor
helm get manifest my-mariadb | grep "300Mi" -A4 -B4
## alles was ich ausgeben kann an Daten aus secrets .
helm get all my-mariadb
```

```
## Hack COMPUTED VALUES anzeigen lassen
## Welche Werte (values) hat er zur Installation verwendet
helm get all my-mariadb | grep -i computed -A 200
```

```
## besser Variante von David
helm get all my-mariadb | sed -n '/COMPUTED/, /HOOKS/p'
```

Tipp: values aus alter revision anzeigen

```
## Beispiel:
helm get values my-mariadb --revision 1
```

Schritt 3: Exercise: Upgrade to new version

Schritt 3.1. Upgrade und resources beibehalten

- Values wurden bereits im vorherigen Schritt angelegt

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 --dry-run=server -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 -f prod/values.yaml
```

```
kubect1 get pods
## kein neuer pod
```

Schritt 3.2 Fehlgeschlagene Installation, wie lösen ?

```
## Schlägt fehle, weil mit dem upgrade bestimmte Felder nicht überschrieben dürfen,
die geändert wurden im Template
```

Lösung

- Deinstallieren (pvc bleibt erhalten auch beim Deinstallieren -> so macht das helm)
- Und wieder installieren in der neuen Version

```
## Frage, ist das pvc noch ?
kubect1 get pvc
## Ja !
```

NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
data-my-mariadb-0	Bound	pvc-575fd652-e0da-46ed-b917-a2c28629e164	8Gi
RWO	do-block-storage	<unset>	66m

```
## alte revisions behalten
helm uninstall my-mariadb --keep-history
kubect1 get pvc
## auch nach der Deinstallation ist der pvc noch da
## Super !!
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 -f prod/values.yaml
```

```
kubectl get pods
helm get values my-mariadb
```

Uninstall

```
helm uninstall my-mariadb
## namespace wird nicht gelöscht
## händisch löschen
kubectl delete ns app-<namenskuerzel>
## crd's werden auch nicht gelöscht
```

Problem: OutOfMemory (OOM-Killer) if container passes limit in memory

- if memory of container is bigger than limit an OOM-Killer will be triggered
- How to fix. Use memory limit in the application too !
 - <https://techcommunity.microsoft.com/blog/appsonazureblog/unleashing-javascript-applications-a-guide-to-boosting-memory-limits-in-node-js/4080857>

Nur fertiges manifest ausgeben ohne Installation

template

Warum ?

- Ich will vorher sehen, wie mein Manifest aussieht, bevor ich es zum Kube-API-Server schicke.

Was macht das ?

- Rendered das Template.

Was macht es nicht ?

- Da er erst nicht an den schickt,
- Überprüft er nicht, ob der Syntax korrekt ist, nur ob das yaml-format eingehalten wird

Beispiel:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
## Kann sehr lang sein
helm -n app-<namenskuerzel> template my-nginx bitnami/nginx --version 19.0.4 | less
helm -n app-<namenskuerzel> template my-nginx bitnami/nginx --version 19.0.4 | grep -A
4 -i ^Kind
```

template --debug

Warum ?

- Zeigt mein template auch an, wenn ein yaml-Einrückungsfehler oder Syntax - fehler da ist.

Beispiel

```
helm -n app-jm template my-nginx bitnami/nginx --version 19.0.4 --debug
```

Informationen aus nicht installierten Helm-Charts bekommen

```
helm show values bitnami/mariadb
helm show values bitnami/mariadb | grep -B 20 -i "image:"
## recommendation -> redirect to file
helm show values bitnami/mariadb > default-values.yaml
```

```
## Zeigt Chart-Definition, Readme usw. (=alles) an
helm show all bitnami/mariadb
```

```
helm show readme bitnami/mariadb
helm show chart bitnami/mariadb
```

```
helm show crds bitnami/mariadb
```

Chart runterladen und evtl. entpacken und bestimmte Version

```
cd
mkdir -p charts
cd charts
```

```
## Vorher müssen wir den Repo-Eintrag anlegen
helm repo add bitnami https://charts.bitnami.com/bitnami

## Lädt die letzte herunter
helm pull bitnami/mariadb

## Lädt bestimmte chart-version runter
helm pull bitnami/mariadb --version 12.1.6
## evtl. entpacken wenn gewünscht
## tar xvf mariadb-12.1.6.tgz

## Schnelle Variante
helm pull bitnami/mariadb --version 12.1.6 --untar
```

Aufräumen von CRD's nach dem Deinstallieren

Schritt 1: repo hinzufügen

```
helm repo add jetstack https://charts.jetstack.io
```

Schritt 2: chart runterladen und entpacken (zum Gucken)

```
helm pull jetstack/cert-manager
ls -la
helm pull jetstack/cert-manager --untar
ls -la
cd cert-manager
ls -la
```

```
cd templates
ls -la crds.yaml
```

Schritt 3: Installieren

```
cd
mkdir cm-values
cd cm-values
nano values.yaml
```

```
crds:
  enabled: true
```

```
helm install cert-manager jetstack/cert-manager --namespace cert-manager-
<namenskuerzel> --create-namespace -f values.yaml
kubectl -n cert-manager-<namenskuerzel> get all
```

CRD's da ?

```
kubectl get crds | grep cert
```

Deinstallieren

```
helm -n cert-manager-<namenskuerzel> uninstall cert-manager
```

CRD's noch da ?

```
kubectl get crds | grep cert
```

CRD's händisch löschen

```
## Variante 1
kubectl delete crd certificaterequests.cert-manager.io certificates.cert-manager.io
challenges.acme.cert-manager.io clusterissuers.cert-manager.io issuers.cert-
manager.io orders.acme.cert-manager.io
```

Helm Charts entwickeln

eigenes helm chart erstellen (Gruppe)

Chart erstellen

```
cd
mkdir my-charts
cd my-charts
```

```
helm create my-app
```

Install helm - chart

```
## Variante 1:  
helm -n my-app-<namenskuerzel> install my-app-release my-app --create-namespace
```

```
## Variante 2:  
cd my-app  
helm -n my-app-<namenskuerzel> install my-app-release . --create-namespace
```

```
kubectl -n my-app-<namenskuerzel> get all  
kubectl -n my-app-<namenskuerzel> get pods
```

Spezial: Umgang mit Einrückungen

Whitespaces meistern mit "-"

Grundlagen

- In Helm (bzw. in Go-Templates) hast du verschiedene Möglichkeiten, den Umgang mit Whitespace (z. B. Leerzeichen, Zeilenumbrüche) zu steuern:
- `{{ ... }}` :
Standardvariante. Lässt den Whitespace außerhalb der geschweiften Klammern unverändert.
- `{{- ... }}` :
Entfernt den Whitespace links (vor) dem Ausdruck.
- `{{ ... -}}` :
Entfernt den Whitespace rechts (nach) dem Ausdruck, aber AUCH Zeilenumbrüche
- `{{- ... -}}` :
Entfernt Whitespace sowohl links als auch rechts des Ausdrucks, aber AUCH Zeilenumbrüche

Exercise Whitespaces

Explanation

- `{{- -> trim on left side`
- `-}}` -> trim on right side / ALSO: new lines
- trim tabs, whitespaces a.s.o. (see ref)

Walkthrough

```
cd  
mkdir -p helm-exercises  
cd helm-exercises
```

```
## When ever we encounter error while parsing yaml, we can use comment !!!  
helm create testenv  
cd testenv/templates  
rm -fR *.yaml
```



```
nano test.yaml
```

```
## "{{23 -}} < {{- 45}}"
```

```
helm template ..  
helm template --debug ..
```

```
## now with new lines  
nano test2.yaml
```

```
## {{23 -}}  
newline here
```

```
helm template ..  
helm template --debug ..
```

Reference:

- https://pkg.go.dev/text/template#hdr-Text_and_spaces

Type - Conversions

Exercise toYaml

Exercise

```
cd  
mkdir -p helm-exercises  
cd helm-exercises  
helm create example-toyaml  
cd example-toyaml  
rm -fR values.yaml  
rm -fR templates/*
```

```
nano templates/configmap.yaml
```

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: {{ .Release.Name }}-config  
data:  
  app-config.yaml: |  
    {{- toYaml .Values.appConfig | nindent 4 }}
```

```
nano values.yaml
```

```
appConfig:  
  server:  
    port: 8080
```

```
  host: "0.0.0.0"
features:
  auth: true
  metrics: true
database:
  user: "admin"
  password: "secret"
  hosts:
    - db1.example.com
    - db2.example.com
```

```
helm template .
```

Ref:

- https://helm.sh/docs/chart_template_guide/function_list/#type-conversion-functions

Flow Control

if

Prepare (if not done yet)

```
cd
mkdir -p helm-exercises
cd helm-exercises
helm create iftest
cd iftest/templates
rm -fR *.yaml
```

Step 2: values-file erweitern

```
rm ../values.yaml
rm -fR tests
rm -fR NOTES.txt
nano ../values.yaml
```

```
## Adjust values.yaml file accordingly
favorite:
  food: PIZZA
  drink: coffee
```

Step 3: Probably the best solution

```
nano cm.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
```

```
data:
  myvalue: "Hello World"
  {{- if eq .Values.favorite.drink "coffee"}}
  {{ "mug: true" }}
  {{- end }}
```

```
helm template ..
```

Step 4: change favorite drin

```
nano ../values.yaml
```

```
## Adjust values.yaml file accordingly
favorite:
  food: PIZZA
  drink: tea
```

```
helm template ..
```

Reference

- https://helm.sh/docs/chart_template_guide/control_structures/

with

Walkthrough

Preparation

```
cd
mkdir -p helm-exercises
cd helm-exercises
helm create with-example
cd with-example/templates
rm -fR *.yaml
```

```
nano ../values.yaml
```

```
## Adjust values.yaml file accordingly
favorite:
  food: PIZZA
  drink: coffee
```

Step 1:

```
nano cm.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  {{- end }}
```

Step 2a: Does not work because scope does not fit

```
nano cm.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  release: {{ .Release.Name }}
  {{- end }}
```

```
helm template --debug ..
```

Step 2b: Solution 1: (Outside with)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  {{- end }}
  release: {{ .Release.Name }}
```

```
helm template --debug ..
```

Step 2c: Changing the scope

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
```

```
{{- with .Values.favorite }}
drink: {{ .drink | default "tea" | quote }}
food:  {{ .food | upper | quote }}
release: {{ $.Release.Name }}
{{- end }}
```

```
helm template --debug ..
```

range

Preparation

```
cd
mkdir -p helm-exercises
cd helm-exercises
helm create range
cd range/templates
rm -f *.yaml
```

Step 1: Values.yaml

```
favorite:
  drink: coffee
  food: pizza
pizzaToppings:
  - mushrooms
  - cheese
  - peppers
  - onions
```

Step 2 (Version 1):

```
## nano cm.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food:  {{ .food | upper | quote }}
  {{- end }}
  toppings: |-
    {{- range .Values.pizzaToppings }}
    - {{ . | title | quote }}
    {{- end }}
```

Step 3 (Version 2 - works as well)

- Accessing the parent scope

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  toppings: |-
    {{- range $.Values.pizzaToppings }}
    - {{ . | title | quote }}
    {{- end }}
  {{- end }}
```

Named Templates

Helm mit gitlab ci/cd

Helm mit gitlab ci/cd ausrollen

Step 1: Import gitlab - repo and pipeline

```
1. Create new repo on gitlab -> do import
https://gitlab.com/jmetzger/training-helm-chart-kubernetes-gitlab-ci-cd.git
```

Step 2: in .gitlab-ci.yml

```
von ---_>
APP_NAME: my-first-app

in ----->
APP_NAME: my-first-app<namenskuerzel>
```

Step 3: Add your KUBECONFIG as Variable (type: File) to Variables

- https://gitlab.com/jmetzger/training-helm-chart-kubernetes-gitlab-ci-cd/-/settings/ci_cd#js-cicd-variables-settings

ochen Metzger / Training Helm Chart Kubernetes Gitlab CI Cd / CI/CD Settings

Maintainer

Developer

Save changes

Project variables

Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked variable feature hides variable values, but is not a guaranteed method to prevent malicious users from accessing variables. [How can I make my variables more secure?](#)

CI/CD Variables </> 1

Key ↑	Value	Environment
KUBECONFIG_SECRET	*****	All (default)

File

> Pipeline trigger tokens

Trigger a pipeline for a branch or tag by generating a trigger token and using it with an API call. The token impersonates the user who created the pipeline.

> Deploy freezes

Add a freeze period to prevent unintended releases during a period of time for a given environment. You must update the deploy freezes added here. [Learn more](#). Specify deploy freezes using [cron syntax](#).

> Job token permissions

Set permissions for the CI/CD job token. The token is used to authenticate the CI/CD job with the Kubernetes API.

Edit variable

TypeFile

EnvironmentsAll (default)

Visibility

Visible

Can be seen in job logs.

Masked

Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet [regular expressions requirements](#).

Masked and hidden

Masked in job logs, and can never be revealed in the CI/CD settings after the variable is saved.

Flags

Protect variable

Export variable to pipelines running on protected branches and tags only.

Expand variable reference

\$ will be treated as the start of a reference to another variable.

Description (optional)ACCESS TO KUBERNETES

The description of the variable's value or usage.

Step 4: Create a pipeline for deployment

```
stages:                # List of stages for jobs, and their order of execution
  - deploy

variables:
  APP_NAME: my-first-app

deploy:
  stage: deploy
  image:
    name: alpine/stages:                # List of stages for jobs, and their order of
execution
  - deploy

variables:
  APP_NAME: my-first-app

deploy:
  stage: deploy
  image:
    name: alpine/k8s:1.31.13
## Important to unset entrypoint
  entrypoint: [""]
  script:
    - ls -la
    - cd; mkdir .kube; cd .kube; cat $KUBECONFIG_SECRET > config; ls -la;
    - cd $CI_PROJECT_DIR; helm upgrade ${APP_NAME} ./charts/my-app --install --
namespace ${APP_NAME} --create-namespace -f ./config/values.yaml
  rules:
```

```

- if: $CI_COMMIT_BRANCH == 'master'
  when: always

## Important to unset entrypoint
  entrypoint: [""]
  script:
    - ls -la
    - cd; mkdir .kube; cd .kube; cat $KUBECONFIG_SECRET > config; ls -la;
    - cd $CI_PROJECT_DIR; helm upgrade ${APP_NAME} ./charts/my-app --install --
namespace ${APP_NAME} --create-namespace -f ./config/values.yaml
  rules:
    - if: $CI_COMMIT_BRANCH == 'master'
      when: always

```

Reference: Example Project (Public)

- <https://gitlab.com/jmetzger/training-helm-chart-kubernetes-gitlab-ci-cd>

Metrics - Server

Metrics - Server mit helm installieren und verwenden

Warum ?

- Es wird ein API bereitgestellt, die Informationen zu den Auslastung von Pods und Nodes sammelt

Installation

```

helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm -n kube-system upgrade --install my-metrics-server metrics-server/metrics-server
--version 3.12.2

```

- Achtung, danach geht es nicht sofort, es dauert einen Momeent bis ich es verwenden kann (geschätzt 5 Minuten)

Verwendung

```

kubectl top pods
## Pods in allen Namespaces
kubectl top pods -A
kubectl top nodes

```

Helmfile

Praxisbeispiel: openDesk mit Helmfile ausrollen

Hintergrund

openDesk (ZenDiS/BMI) ist kein einzelnes Helm-Chart, sondern 35+ Charts (Keycloak/Nubus als IAM, Nextcloud, OX App Suite, OpenProject, XWiki, Element/Synapse, Jitsi, Collabora, CryptPad, Mailstack, Datenbanken, ...), orchestriert ueber [Helmfile](#). Kein `helm install opendesk` - stattdessen `helmfile apply -e <environment> -n <namespace>` .

Dieses Dokument beschreibt den Rollout auf DigitalOcean als Referenz fuer aehnliche Helmfile-basierte Multi-Chart-Deployments. **Ergebnis vorweg:** Auf einem Managed-Cluster (DOKS) blieb der LDAP-Server (siehe Troubleshooting) instabil - der erfolgreiche, stabile Rollout lief am Ende auf einem selbstverwalteten 3-Node-kubeadm-Cluster (Kubernetes 1.32.13 auf Ubuntu-24.04- Droplets). Grund und Fix sind unten dokumentiert.

Voraussetzungen

Tool	Version	Hinweis
Kubernetes	>= 1.24, CNCF-zertifiziert	getestet mit 1.35.5 (DOKS, LDAP instabil) und 1.32.13 (kubeadm, stabil - siehe Troubleshooting)
Helm	>= 3.17.3, NICHT 3.18.0 / 3.20.1	bekannte Helm-Bugs in diesen Versionen
Helmfile	>= 1.0.0	
Helm Diff Plugin	>= 3.11.0	helm plugin install
yq	>= 4.52.4	
Ingress Controller	haproxy-ingress.github.io (Default seit openDesk >= 1.13)	ingress-nginx geht technisch auch (Override noetig), ist aber upstream deprecated und nicht mehr der getestete Pfad
cert-manager	mit CRDs	fuer Zertifikate
Volume Provisioner	ReadWriteOnce (min.)	z.B. do-block-storage

Hardware-Minimum laut openDesk-Doku fuer eine Eval-Instanz: 12 CPU-Cores, 32 GB RAM. In der Praxis (siehe Troubleshooting unten) sollte man deutlich mehr RAM einplanen.

Ablauf

Schritt 1: Ingress-Controller und cert-manager installieren

openDesk >= v1.13 verwendet standardmaessig `ingressClassName: haproxy` - das sollte man auch so lassen, nicht auf ingress-nginx umsteigen. Ingress-nginx laesst sich zwar technisch per Override anbinden (Erfahrung aus der Praxis: DNS, Zertifikate und Routing funktionieren grundsaeztlich auch damit), es ist aber upstream deprecated und **nicht der von openDesk getestete Pfad** - man weicht damit unnoetig vom Standard-Setup ab, ohne einen echten Vorteil zu haben.

```
helm repo add haproxy-ingress https://haproxy-ingress.github.io/charts
helm upgrade -i haproxy-ingress haproxy-ingress/haproxy-ingress \
  -n haproxy-ingress --create-namespace \
  --set controller.ingressClass=haproxy \
  --set controller.ingressClassResource.enabled=true \
  --set controller.ingressClassResource.default=true \
  --set controller.config.config-global="tune.bufsize 65536\ntune.http.maxhdr 256"
```

```
helm repo add jetstack https://charts.jetstack.io
helm upgrade -i cert-manager jetstack/cert-manager \
```

```
-n cert-manager --create-namespace --set crds.enabled=true
```

Danach einen `ClusterIssuer` fuer Let's Encrypt anlegen (HTTP-01, passend zur Ingress-Klasse):

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: <deine-email>
    privateKeySecretRef:
      name: letsencrypt-prod-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: haproxy
```

Schritt 2: DNS

openDesk braucht ein Wildcard-DNS auf die Ingress-LoadBalancer-IP - jede Komponente bekommt eine eigene Subdomain (z.B. `office.<domain>`, `id.<domain>`, `pad.<domain>`).

```
## LoadBalancer-IP ermitteln
kubectl -n haproxy-ingress get svc haproxy-ingress -o
jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

DNS-Eintraege: `<domain>` und `*.<domain>` als A-Record auf diese IP.

Schritt 3: Repo klonen und konfigurieren

```
git clone https://gitlab.opencode.de/bmi/opendesk/deployment/opendesk.git
cd opendesk
git checkout v1.16.1
```

Eigene Einstellungen kommen als `.yaml.gotmpl` -Dateien in `helmfile/environments/dev/` (nicht die Default-Dateien in `helmfile/environments/default/` anfassen) - z.B. fuer Ressourcen-Anpassungen (siehe Troubleshooting). Der Ingress muss dafuer nicht angefasst werden, da `haproxy` bereits Default ist.

Schritt 4: Deployen

```
export DOMAIN=meine-domain.de
export MASTER_PASSWORD=<sicheres-passwort-per-openssl-rand-generiert>

helmfile apply -e dev -n opendesk
```

`MASTER_PASSWORD` ist Pflicht (kein Default mehr seit v1.14) und wird intern per `derivePassword` fuer alle Sub-Secrets (Datenbanken, LDAP, OX AppSuite, ...) verwendet - ein Passwort reicht fuer die komplette Eval-Instanz.

Sicherheitshinweis: `MASTER_PASSWORD` niemals im Klartext in Chat/Terminal-History landen lassen - per `openssl rand` generieren, SOPS-verschlusselt ablegen (`.env.enc`), zur Laufzeit per `sops --decrypt` in eine Shell-Variable einlesen.

Schritt 5: Default-User importieren

Ueber die UDM-REST-API mittels Docker-Container:

```
docker run --rm registry.opencode.de/bmi/opendesk/components/platform-
development/images/user-import:3.0.0 \
  ./user_import_udm_rest_api.py \
  --import_domain ${DOMAIN} \
  --udm_api_password ${MASTER_PASSWORD} \
  --set_default_password <user-pw> \
  --import_filename template.ods \
  --create_admin_accounts True
```

Troubleshooting (Praxis-Erfahrungen)

helm upgrade schlaegt fehl mit jobs.batch "<name>--N" not found

Trat bei `opendesk-migrations-pre` und `opendesk-nextcloud-management` auf, wenn `helmfile apply` nach einem Fehler erneut ausgefuehrt wird. Ursache: die ConfigMap `migrations-status` im Ziel-Namespace haelt den Migrations-Status fest und verwirrt den Job-Hook bei einem Retry ("Current Stage PRE is the same as PRE - this is suspicious").

Fix: ConfigMap loeschen und erneut `helmfile apply` ausfuehren.

```
kubectl -n opendesk delete configmap migrations-status
```

ums-ldap-server-primary crasht mit OOMKilled (ungeloest auf DOKS)

Der Nubus-LDAP-Server (Univention-basiert, Chart `nubus`) kann beim Start extrem viel Speicher allozieren - in einem Test: Wachstum von ~100MB auf ueber 11 GB in unter 3 Sekunden, live per `kubectl exec ... cat /sys/fs/cgroup/memory.current` verfolgt. Der Standard-Wert des Charts (`nubusDevelopmentResources`) liegt bei nur 1Gi Limit; Univention selbst nennt fuer Produktion 2048Mi als Default - der Hersteller geht also nicht von einem hohen Speicherbedarf aus.

Folgende Ursachen wurden systematisch ausgeschlossen:

- **Zu wenig RAM:** Getestet mit Limits von 1Gi bis 40Gi (dedizierter `m-8vcpu-64gb` -Node) - das Wachstum floppt bei keiner Grenze ab, sondern waechst kontinuierlich weiter (live gemessen: 24GB -> 38GB in ~30 Sekunden), bis es die jeweilige Grenze erreicht und gekillt wird. Kein einmaliger Init-Peak, der irgendwann plateauet.
- **Alte/korrumperte Daten:** Frische, leere PVCs (nach komplettem `helm uninstall` inkl. PVC-Loeschung) zeigen dasselbe Verhalten. Die eigentlichen LDAP-Datenverzeichnisse (`/var/lib/univention-ldap/ldap/`) blieben ueber alle Versuche hinweg leer, der Translog wuchs nie ueber 32KB - das Problem haengt nicht von angesammeltem Datenvolumen ab.
- **Chart-/openDesk-Versions-Regression:** Mit openDesk v1.16.1 (nubus 1.20.1) UND v1.14.3 (nubus 1.19.1) reproduziert - identisches Verhalten in zwei unabhängigen Versionskombinationen.

Ein echter (und notwendiger) Bug wurde nebenbei trotzdem gefunden: `ums-ldap-notifier` und `ums-ldap-server-primary` teilen sich ein RWO-Volume und muessen daher zwingend auf demselben Node laufen (der Chart definiert dafuer eine `podAffinity`). Wird der `ldap-server-primary` -Pod manuell neu gestartet (z.B. `kubectl delete pod`), kann der Scheduler ihn auf einen anderen Node setzen als den Notifier -> `Multi-Attach error for volume ... already used by pod(s) ums-ldap-notifier-0`. Fix: betroffenen Notifier-Pod ebenfalls loeschen, damit er sich per Required-Affinity wieder neben dem Primary einordnet.

```
kubectl -n opendesk delete pod ums-ldap-notifier-0
```

Geloest. Ursache war ein Kubernetes-cgroup-v2-Verhalten: `memory.oom.group` (seit K8s 1.28 aktiv) kllt bei cgroup v2 den **kompletten Container**, sobald irgendein einzelner Prozess darin kurzzeitig OOM geht - auch bei einer voellig harmlosen, transienten Speicherspitze, wie sie LMDB (das Speicherformat von OpenLDAP) beim Start durch memory-mapped I/O erzeugt. Seit K8s 1.32 gibt es das Kubelet-Flag `singleProcessOOMKill`, das genau das verhindert (nur der einzelne Prozess wird gekillt, nicht der ganze Container) - Default ist aber weiterhin "aus" fuer cgroup v2.

Da DOKS als Managed-Service keine Kubelet-Konfiguration erlaubt, war das dort nicht loesbar. Auf einem selbstverwalteten kubeadm-Cluster (3 Droplets, Kubernetes 1.32.13, `kubeadm init` mit einer `KubeletConfiguration` inkl. `singleProcessOOMKill: true`) lief `ums-ldap-server-primary` sofort stabil: tatsaechlicher Speicherbedarf liegt bei ~86 MB (!) statt der zuvor gemessenen 40GB+. Bestaetigt der urspruenglichen Vermutung: der "hohe Speicherbedarf" war nie real, sondern ein Kill-Artefakt der cgroup-v2-Policy.

```
## kubeadm-config.yaml (Auszug) - vor "kubeadm init" setzen, gilt dann automatisch
## fuer alle spaeter beitretenden Nodes (aus der kubelet-config ConfigMap)
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
singleProcessOOMKill: true
```

Praktische Konsequenz: Fuer openDesk (bzw. jeden anderen LMDB/mmap-lastigen Workload) auf Kubernetes >= 1.28 mit cgroup v2 ist ein Managed-Kubernetes-Anbieter, der keine Kubelet-Config erlaubt (wie DOKS), ungeeignet - zumindest ohne diesen Fix upstream im Chart/Image. Alternativen: selbstverwalteter Cluster (kubeadm/kubespray, wie hier), oder ein Managed-Angebot, das Kubelet-Extra-Args/-Config zulaesst.

helmfile apply erneut ausfuehren startet Job-Hooks (z.B. Schema-Ladejobs) neu

Jobs, die als Helm-Hooks definiert sind (z.B. `ums-stack-data-ums`, `ums-provisioning-register-consumers`, `ums-keycloak-bootstrap`), laufen bei **jedem** `helmfile apply` erneut, sobald der zugehoerige Release ein Diff hat (auch ein voellig harmloses, wie ein Secret-Rotationswert). Das kann fuer ein paar Minuten kurzzeitig 401/503-Fehler bei abhaengigen Consumern verursachen (z.B. `ums-portal-frontend`, `ums-provisioning-udm-transformer`, `ox-connector`), bis die Hooks durchgelaufen sind und die Subscriptions neu registriert wurden. Kein Bug, nur bei einem erneuten `helmfile apply` auf ein bereits laufendes System zu beachten - i.d.R. loest es sich innerhalb weniger Minuten von selbst; falls nicht, betroffene Pods einmal `kubectl delete pod` neu starten.

Geloeschte PVCs haengen in Terminating

Auf DigitalOcean kann das automatische Detachen eines Block-Storage-Volumes vom Node mehrere Minuten dauern oder haengen bleiben. Kein K8s-Fehler, sondern eine asynchrone Cloud-Operation. Falls es nach 5+ Minuten immer noch haengt: Volume manuell per `doctl` detachen, dann die `VolumeAttachment` -Objekte in Kubernetes loeschen.

```
doctl compute volume-action detach <volume-id> <droplet-id>
kubectl delete volumeattachment <name>
```

Fazit

Ein Helmfile-Deployment dieser Groessenordnung (35+ Charts, SSO/LDAP/Datenbanken) ist deutlich komplexer als ein einzelnes `helm install` und bringt eigene Fehlerklassen mit sich: Job-Hook- State bei Retries, Cross-Pod- Volume-Affinity und - am aufwendigsten zu diagnostizieren - Kubernetes-Plattform-Verhalten (cgroup v2 OOM-Handling), das mit bestimmten Workloads (memory-mapped I/O wie LMDB) kollidiert.

Auf einem DigitalOcean-Managed-Kubernetes-Cluster (DOKS) blieb der LDAP-Server trotz umfangreicher Diagnose (Ressourcen, PVCs, Node-Affinity, Chart-Versionen) instabil, weil DOKS als Managed-Service keinen Zugriff auf die Kubelet-Konfiguration erlaubt. Der Wechsel auf einen selbstverwalteten kubeadm-Cluster (3 Droplets, `singleProcessOOMKill: true`) - genau der Weg, gegen den openDesk selbst testet (kubespray-basiert) - hat das Problem sofort behoben.

Praxis-Lehre: Bei mysteriösen OOM-Kills mit scheinbar absurd hohem, nicht plateauendem Speicherbedarf lohnt sich der Blick auf die cgroup-v2-OOM-Policy des Clusters, bevor man Limits immer weiter hochdreht - insbesondere bei Workloads mit memory-mapped I/O (LMDB, RocksDB, u.ae.) und auf Managed-Kubernetes-Angeboten ohne Kubelet-Zugriff.

Für ein Trainings-Lab ohne diesen Anspruch lohnt sich ggf. trotzdem der Blick auf die leichtgewichtigeren Alternativen (K3s-Guide, SCS-Dokumentation von openDesk).

Endergebnis: Auf dem kubeadm-Cluster läuft openDesk (26 Helm-Releases, kein einziger dauerhaft fehlgeschlagen) stabil, das Portal ist unter der konfigurierten Domain per HTTPS mit gültigem Let's-Encrypt-Zertifikat erreichbar, `ums-ldap-server-primary` blieb über Stunden durchgehend `Ready` ohne einen einzigen Neustart.

helm - Dokumentation

Helm Documentation

- <https://helm.sh/docs/>

Grundlagen

Feature / No-Features von Helm

- Sortiert, die Manifeste bzw. Objekte bereits automatisch in der richtigen Reihenfolge für das Anwenden (apply) gegen den Server (Kube-API-Server)

Which order is it ?

- see also Internals [Helm Sorting Objects](#)

Tipps & Tricks

kubernetes manifests mit privatem Repo

Exercise

```
mkdir -p manifests
cd manifests
mkdir private-repo
cd private-repo
```

```
kubectl create secret docker-registry regcred --docker-server=registry.do.t3isp.de \
--docker-username=11trainingdo --docker-password=<sehr-geheim> --dry-run=client -o
yaml > 01-secret.yaml
```

```
kubectl create secret generic mariadb-secret --from-
literal=MARIADB_ROOT_PASSWORD=11abc432 --dry-run=client -o yaml > 02-secret.yaml
```

```
nano 02-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: private-reg
spec:
  containers:
  - name: private-reg-container
    image: registry.do.t3isp.de/mariadb:11.4.5
    envFrom:
    - secretRef:
        name: mariadb-secret
    imagePullSecrets:
    - name: regcred
```

```
kubectl apply -f .
kubectl get pods -o wide private-reg
kubectl describe pods private-reg

### helm chart mit images auf privatem Repo

### Walkthrough
```

```
cd mkdir -p manifests cd manifests mkdir nginx-values cd nginx-values mkdir prod cd prod nano values.yaml
```

```
global: security: allowInsecureImages: true
```

```
image: registry: "registry.do.t3isp.de" repository: nginx
```

tag: 1.27.4

```
pullSecrets: - regcred-do
```

```
extraDeploy:
```

- apiVersion: v1 data: .dockerconfigjson: kind: Secret metadata: name: regcred-do type: kubernetes.io/dockerconfigjson

```
cd cd manifests/nginx-values helm upgrade --install my-nginx bitnami/nginx -f prod/values.yaml
```

```
## Helm-Befehle und -Funktionen
```

```
### Repo einrichten
```

```
helm repo list helm repo add bitnami https://charts.bitnami.com/bitnami helm repo remove bitnami helm repo update
```

```
### Suche in Repo und Artifacts Hub
```

```
### Suche im hub
```

```
helm search hub mariadb
```

Zeige kompletten Zeilen an ohne abzuschneiden

```
helm search hub mariadb --max-col-width=0
```

```
### Suche im Repo
```

Suche nach allen Charts, die mariadb im Namen oder der Beschreibung tragen

```
helm search repo mariadb
```

Zeige alle Version von charts an, die mit bitnami/mariadb beginnen

```
helm search repo bitnami/mariadb --versions
```

```
### Anzeigen von Informationen aus dem Chart von Online
```

```
helm show values bitnami/mariadb helm show values bitnami/mariadb | grep -B 20 -i "image:"
```

recommendation -> redirect to file

```
helm show values bitnami/mariadb > default-values.yaml
```

Zeigt Chart-Definition, Readme usw. (=alles) an

```
helm show all bitnami/mariadb
```

```
helm show readme bitnami/mariadb helm show chart bitnami/mariadb
```

```
helm show crds bitnami/mariadb
```

```
### Upgrade und auftretende Probleme
```

```
### Die wichtigsten Repo-Befehle
```

```
helm repo list helm repo add bitnami https://charts.bitnami.com/bitnami helm repo remove bitnami helm repo update
```

```
## Struktur von Helm - Charts
```

```
### Überblick
```

```
### Komponenten von Helm-Charts
```

```
#### Chart.yml
```

```
#### Chart.lock (wird automatisch generiert)
```

```
#### templates/
```

```
##### _helper.tpl
```

- * Enthält snippet die mit include oder templates inkludiert werden können
- * Konvention der Snippets mit define ChartName.Eigenschaft z.B. botti.fullname

```
##### NOTES.txt
```

- * Wird ausgegeben, nachdem das Chart installiert wurde
- * oder:

after installation

helm install my-botti -n my-application --create-namespace botti

```
helm get -n my-application notes my-botti
```

```
#### charts/
```

- * Hier werden die abhängigen charts runtergeladen und als .tgz


```
## Grundlagen Helm-Charts

### Testumgebung und Spaces (2 Themen)

### Explanation

* {{- -> trim on left side
* -}} -> trim on right side / ALSO: new lines
* trim tabs, whitespaces a.s.o. (see ref)

### Walkthrough
```

```
cd mkdir -p helm-exercises cd helm-exercises
```

When ever we encounter error while parsing yaml, we can use comment !!!

```
helm create testenv cd testenv/templates rm -fR *.yaml
```

```
nano test.yaml
```

```
"{{23 -}} < {{- 45}}"
```

```
helm template .. helm template --debug ..
```

now with new lines

```
nano test2.yaml
```

```
{{23 -}}
```

```
newline here
```

```
helm template .. helm template --debug ..
```

```
### Reference:

* https://pkg.go.dev/text/template#hdr-Text_and_spaces

## Erstellen von Helm-Charts

### Erstellen eines Guestbooks

### Step 1: Create namespace and structure of helm chart
```

cd

helm create guestbook

now we have in folder "guestbook"

charts/

Chart.yaml

templates

values.yaml

```
### Step 2: Explore templates folder and cleanup
```

cd templates ls -la rm -fR tests

```
### Step 3: Explore the Chart.yaml
```

cd .. cat Chart.yaml

type: Application or Library # please explain !

dependencies - what other charts are needed - we will download them by helm command and they will be put in the charts - folder

```
### Step 4: Add redis as dependency
```

find the redis chart

```
helm search hub --max-col-width=0 redis | grep bitnami
```

adding the repo for bitnami

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

now find the available versions (these are the chart versions)

```
helm search repo redis --versions
```

```
nano Chart.yaml
```

now add the dependency-block at the end of the file

dependencies:

- name: redis version: "17.14.x" # quotes are important here repository: <https://charts.bitnami.com/bitnami>

Save the file and leave nano:

STRG + o + RETURN -> then -> STRG + x

```
cd .. helm dependency update guestbook
```

explore the newly populated folder

```
cd guestbook/charts ls -la cd ../../
```

```
### Step 5: Modifying the values.yaml file
```

the version might have changed since i wrote this / adjust

```
helm show values charts/redis-17.14.5.tgz
```

what are the service name of the redis leader and the redis follower

```
helm show values charts/redis-17.14.5.tgz | grep -B 4 -i fullnameoverride
```

the service names need to be adjusted, add the following to the values.yaml

The guestbook - application needs the redis - services called. redis-leader and redis-follower

```
cd cd guestbook nano values.yaml
```

add at the end of the file

```
redis: fullnameOverride: redis
```

enable unauthorized access to redis

```
usePassword: false
```

Disable AOF persistence

```
configmap: |- appendonly no
```

save file and exit

```
STRG + o + ENTER -> then -> STRG + x
```

now check, if this really worked

```
cd cd guestbook helm template . | grep -A 20 master/service
```

```
### Setting the right repo and the right version
```

```
cd cd guestbook cat templates/deployment.yaml
```

Welche Version brauche ich ? <https://kubernetes.io/docs/tutorials/stateless-application/guestbook/#creating-the-guestbook-frontend-deployment>

Stand 2023-08-08

gcr.io/google_samples/gb-frontend:v5

nano Chart.yaml

korrigieren

appVersion: "v5"

nano values.yaml

image: repository: gcr.io/google_samples/gb-frontend

```
### Step 6: Changing LoadBalancer to NodePort
```

nano values.yaml

service: type: NodePort port: 80

```
### Step 7: Installing helm chart
```

helm install my-guestbook guestbook -n jochen --create-namespace kubectl -n jochen get all

```
### Reference:
```

```
* https://kubernetes.io/docs/tutorials/stateless-application/guestbook/
```

```
### Hooks für Guestbook erstellen
```

```
### Step 1:
```

cd mkdir guestbook/templates/backup touch guestbook/templates/backup/persistentVolume-claim.yaml touch guestbook/templates/backup/job.yaml

```
### Step 2: persistentvolumeclaim.yaml und job bevölkern
```

nano guestbook/templates/backup/persistentVolume-claim.yaml

```
{{- if .Values.redis.master.persistence.enabled }} apiVersion: v1 kind: PersistentVolumeClaim metadata: name: redis-data-{{ .Values.redis.fullnameOverride }}-master-0-backup-{{ sub .Release.Revision 1 }} labels: {{- include "guestbook.labels" . | nindent 4 }} annotations: "helm.sh/hook": pre-upgrade "helm.sh/hook-weight": "0" spec: accessModes: - ReadWriteOnce resources: requests: storage: {{ .Values.redis.master.persistence.size }} {{- end }}
```

nano guestbook/templates/backup/job.yaml

```
{{- if .Values.redis.master.persistence.enabled }} apiVersion: batch/v1 kind: Job metadata: name: {{ include "guestbook.fullname" . }}-backup labels: {{- include "guestbook.labels" . | nindent 4 }} annotations: "helm.sh/hook": pre-upgrade "helm.sh/hook-delete-policy": before-hook-creation, hook-succeeded "helm.sh/hook-weight": "1" spec: template: spec: containers: - name: backup image: redis:alpine3.11 command: ["/bin/sh", "-c"] args: ["redis-cli -h {{ .Values.redis.fullnameOverride }}-master save && cp /data/dump.rdb /backup/dump.rdb"] volumeMounts: - name: redis-data mountPath: /data - name: backup mountPath: /backup restartPolicy: Never volumes: - name: redis-data persistentVolumeClaim: claimName: redis-data-{{ .Values.redis.fullnameOverride }}-master-0 - name: backup persistentVolumeClaim: claimName: redis-data-{{ .Values.redis.fullnameOverride }}-master-0-backup-{{ sub .Release.Revision 1 }} {{- end }}
```

```
### Step 3: pre-rollback hook erstellen
```

```
mkdir guestbook/templates/restore touch guestbook/templates/restore/job.yaml
```

nano guestbook/templates/restore/job.yaml

```
{{- if .Values.redis.master.persistence.enabled }} apiVersion: batch/v1 kind: Job metadata: name: {{ include "guestbook.fullname" . }}-restore labels: {{- include "guestbook.labels" . | nindent 4 }} annotations: "helm.sh/hook": pre-rollback "helm.sh/hook-delete-policy": before-hook-creation, hook-succeeded spec: template: spec: containers: - name: restore image: redis:alpine3.11 command: ["/bin/sh", "-c"] args: ["cp /backup/dump.rdb /data/dump.rdb && redis-cli -h {{ .Values.redis.fullnameOverride }}-master debug restart || true"] volumeMounts: - name: redis-data mountPath: /data - name: backup mountPath: /backup restartPolicy: Never volumes: - name: redis-data persistentVolumeClaim: claimName: redis-data-{{ .Values.redis.fullnameOverride }}-master-0 - name: backup persistentVolumeClaim: claimName: redis-data-{{ .Values.redis.fullnameOverride }}-master-0-backup-{{ .Release.Revision }} {{- end }}
```

```
### Reference
```

```
* https://helm.sh/docs/topics/charts\_hooks/
```

```
### Dependencies/Abhängigkeiten herunterladen
```

```
### Voraussetzung:
```

```
* Dependencies sind in Chart.yml eingetragen
```

```
* Achtung: Version ist die Version des Charts nicht der App !!!
```

```
### Das 1. Mal
```

1. Alle Abhängigkeiten werden in Form von .tgz - Archiven heruntergeladen

```
-> in das charts - Verzeichnis
```

2. Eine Chart.lock - datei wird erstellt. (hält den aktuellen Stand fest)

helm dependency update \$CHART_PATH

botti erklärt sich gleich unten im Walkthrough

```
helm dependency update botti
```

```
### Das 2. Mal (wenn Chart.lock vorhanden, aber charts/ muss nicht da sein
```

```
helm dependency build botti
```

```
### List all dependencies
```

```
helm dependency list botti
```

```
### Walkthrough
```

```
cd helm create botti
```

```
cd botti
```

add dependency

```
nano Chart.yml
```

at the end of the file add

After that save and exit STRG + O + ENTER , STRG + X

Update to download dependancies

```
cd .. helm dependency update botti cd botti/charts ls -la cd ../../
```

Add repo to be able to do helm dependency build

```
rm -fR botti/charts
```

Chart.lock needs to be there

```
ls -la botti/Chart.lock
```

Add repo / needs to be there, otherwise

```
helm repo add bitnami https://charts.bitnami.com/bitnami helm dependency build botti
```

```
### Einfaches Testen

### Input Validierung innerhalb von templates

### Walkthrough
```

```
cd helm create inputtest cd inputtest cd templates/ rm d* h* i* servicea* rm -fR tests
```

nano service.yaml mit folgendem Inhalt

```
apiVersion: v1 kind: Service metadata: name: {{ include "inputtest.fullname" . }} labels: {{- include "inputtest.labels" . |
nindent 4 }} spec: {{- $serviceType := list "ClusterIP" "NodePort" }} {{- if has .Values.service.type $serviceType }} type: {{
.Values.service.type }} {{- else }} {{- fail "value 'service.type' must be either 'ClusterIP' or 'NodePort'" }} {{- end }} ports: -
port: {{ .Values.service.port }} targetPort: http protocol: TCP name: http selector: {{- include "inputtest.selectorLabels" . |
nindent 4 }}
```

```
cd cd inputtest nano values.yaml
```

```
service: type: nodePorty # written wrong port: 80
```

```
cd helm template --debug inputtest
```


and eventually also test against server

helm template inputtest --validate

```
### Advanced Testing mit chart-testing

### Reference

* https://github.com/helm/chart-testing/
* https://github.com/helm/chart-testing/blob/main/doc/ct_install.md

### Chart auf github veröffentlichen

### Prep
```

Create new public repo with README.md Go to Settings -> Pages -> an enable for branch "main" git clone the repo locally

```
### Locally pack, index and upload it.
```

git clone <https://github.com/jmetzger/chart-test.git>

guestbook must be present as folder with charts

helm package guestbook cp guestbook-0.1.0.tgz chart-test/ helm repo index chart-test/ git add . git commit -m "initial release" git push -u origin main

```
### Work with it
```

helm repo add githubrepo <https://jmetzger.github.io/chart-test/> helm search repo guestbook helm repo list helm pull githubrepo/guestbook

```
## Sicherheit von helm-Chart

### Grundlagen / Best Practices

* https://sysdig.com/blog/how-to-secure-helm/

### Security Encrypted Passwords in helm
```

```
### Reference:
```

```
* https://www.thorsten-hans.com/encrypted-secrets-in-helm-charts/  
* https://github.com/jkroepke/helm-secrets
```

```
### Alternative: SealedSecrets
```

```
* https://dev.to/timtsoitt/argo-cd-and-sealed-secrets-is-a-perfect-match-1dbf
```

```
## Testing in Helm-Charts
```

```
### Testing in/von helm - charts
```

```
### Walkthrough
```

helm create demo helm install demo demo helm test demo

```
### Reference
```

```
* https://helm.sh/docs/topics/chart_tests/
```

```
## Durchführung von Upgrades und Rollbacks von Anwendungen
```

```
## Helm in Continuous Integration / Continuous Deployment (CI/CD) Pipelines
```

```
## Tipps & Tricks
```

```
### Set namespace in config of kubectl
```

kubectl create ns mynamespace kubectl config set-context --current --namespace=mynamespace

```
### Create Ingress Redirect
```

cd helm create testprojekt cd testprojekt cd templates

mkdir routes/ cd routes nano 01-redirect.yaml

```
### Schritt 1: Mit der Basis anfangen
```

apiVersion: networking.k8s.io/v1 kind: Ingress metadata: annotations: nginx.ingress.kubernetes.io/permanent-redirect:
<https://www.google.de> nginx.ingress.kubernetes.io/permanent-redirect-code: "308" creationTimestamp: null name:
destination-home namespace: my-namespace spec: rules:

- host: web.training.local http: paths:
 - backend: service: name: http-svc port: number: 80 path: /source pathType: ImplementationSpecific

```
### Schritt 2: values - file mit eigenen Werten ergänzen (Default - Werte)
```

cd ../..

nano values.yaml

Zeilen ergänzt.

Achtung: Eigenschaft UNBEDINGT ! ohne "-"

myRedirect: url: "<http://www.google.de>" code: 302

```
### Schritt 3: Variablen aus values in template einbauen
```

cd templates/routes

nano 01-redirect.yaml

Neue Fassung: Alle Änderungen beginnen mit Platzhalter - Zeichen {{

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: annotations: nginx.ingress.kubernetes.io/permanent-redirect:
{{ .Values.myRedirect.url }} nginx.ingress.kubernetes.io/permanent-redirect-code: {{ .Values.myRedirect.code | quote }}
creationTimestamp: null name: destination-home namespace: my-namespace spec: rules:
```

- host: web.training.local http: paths:
 - backend: service: name: http-svc port: number: 80 path: /source pathType: ImplementationSpecific

```
### Schritt 4: Test mit Default - Werten aus values.yaml
```

helm template ../..

achten auf ausgaben von Ingress

helm template ../.. | grep -A 40 "kind: Ingress"

```
### Schritt 5: Default - Werte überschreibung für Produktion mit speziellen prod-
values.yaml (Name beliebig)
```

Empfehlung: ausserhalb des Charts anlegen

```
cd nano prod-values.yaml
```

```
myRedirect: url: "http://www.stiftung-warentest.de"
```

Testen wie folgt

```
helm template -f prod-values.yaml testprojekt
```

oder aber auch testen mit validate

```
helm template --validate -f prod-values.yaml testprojekt
```

oder aber direkt release installation

```
helm install --dry-run -f prod-values.yaml testprojekt
```

```
### Helm Charts - Development - Best practices

* https://helm.sh/docs/howto/charts\_tips\_and\_tricks/

## Integration mit anderen Tools

### yamllint für Syntaxcheck von yaml - Dateien
```

```
apt install -y yamllint
```

```
## Troubleshooting und Debugging

### helm template --validate - gegen api-server testen

### How ?
```

```
helm template guestbook --validate
```